CO2 AT3 – Debugging and Optimization Report

Debugging Insertion Sort Code – Key Overwritten Before Insertion Completes

1. Objective

The objective of this debugging task is to identify and correct the error in an **Insertion Sort** implementation where the key variable is overwritten during the shifting process.

The main objectives are to:

- Understand the role of the key variable in Insertion Sort.
- Identify why overwriting key causes incorrect sorting.
- Debug the program systematically.
- Ensure that larger elements are shifted one position to the right.
- Preserve the original key until its correct position is found.
- Improve the readability and consistency of the implementation.
- Analyze the time and space complexity.
- Validate the corrected implementation using multiple test cases.

2. Problem Identification

2.1 Given Code

The faulty code is:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            key = arr[j]       # ERROR
            arr[j + 1] = arr[j]
            j -= 1
```

```
        arr[j + 1] = key


    return arr
```

The main problem is:

key = arr[j]

inside the shifting loop.

3. Understanding the Role of key

In Insertion Sort, key represents the element currently being inserted into the already sorted portion of the array.

For example:

[10, 20, 30, 15, 40]

```
          ↑
         key
```

Here:

key = 15

The algorithm must preserve 15 while shifting larger elements:

[10, 20, 30, 30, 40]

Then it inserts the original key:

[10, 15, 20, 30, 40]

Therefore, **the key must remain unchanged during the shifting process**.

4. Root Cause Analysis

The root cause is the statement:

key = arr[j]

The key should never be overwritten.

Instead, the larger element should be shifted:

arr[j + 1] = arr[j]

The correct operation inside the loop is therefore:

while j >= 0 and arr[j] > key:

    arr[j + 1] = arr[j]

    j -= 1

The original key remains stored safely in:

key

until the correct insertion position is identified.

5. Why the Original Code Fails

Consider:

[5, 8, 3]

During the second iteration:

key = 3

The sorted portion is:

[5, 8]

Since:

8 > 3

the algorithm should shift 8:

[5, 8, 8]

Then:

5 > 3

so it should shift 5:

[5, 5, 8]

Finally, insert:

3

giving:

[3, 5, 8]

What the Faulty Code Does

The faulty statement:

key = arr[j]

changes:

key = 3

to:

key = 8

The original value 3 is lost.

On the next iteration, key may become 5.

Eventually, the wrong value is inserted.

Thus, the algorithm can produce an incorrect result.

6. Debugging Methodology

A systematic debugging process was followed.

Step 1 – Identify the Key

The code correctly initializes:

key = arr[i]

This stores the element that must be inserted.

Step 2 – Identify the Sorted Portion

At iteration i, the elements from:

0 to i-1

are assumed to already be sorted.

Step 3 – Inspect the Shifting Condition

The condition:

while j >= 0 and arr[j] > key:

is correct.

It identifies elements that are larger than the key.

Step 4 – Inspect the Shifting Operation

The faulty line is:

key = arr[j]

This overwrites the original key.

It should instead shift the larger element:

arr[j + 1] = arr[j]

Step 5 – Preserve the Key

The key should remain unchanged throughout the while loop.

Therefore:

key = arr[i]

is assigned only once for each outer-loop iteration.

Step 6 – Insert the Key

After all larger elements have been shifted:

arr[j + 1] = key

places the original element into its correct position.

7. Corrected Algorithm

The corrected algorithm is:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Shift larger elements one position to the right
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
```

```
        j -= 1

    # Insert the original key at its correct position
    arr[j + 1] = key

return arr
```

8. Algorithm / Procedure

Step 1

Start from the second element:

[
i=1
]

because a single element is already sorted.

Step 2

Store the current element:

[
key=A[i]
]

Step 3

Set:

[
j=i-1
]

Step 4

Compare:

[
A[j]>key
]

Step 5

If true, shift the larger element:

[

A[j+1]=A[j]
]

Step 6

Move left:

[
j=j-1
]

Step 7

Repeat until:

- j < 0, or
- A[j] <= key.

Step 8

Insert the preserved key:

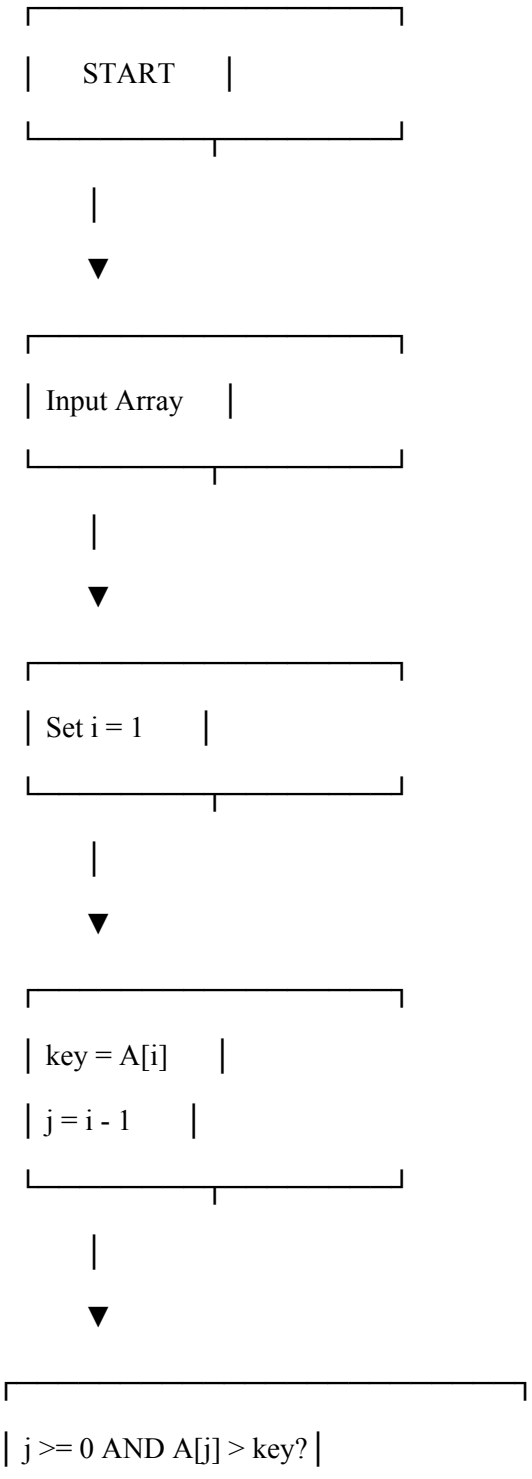
[
A[j+1]=key
]

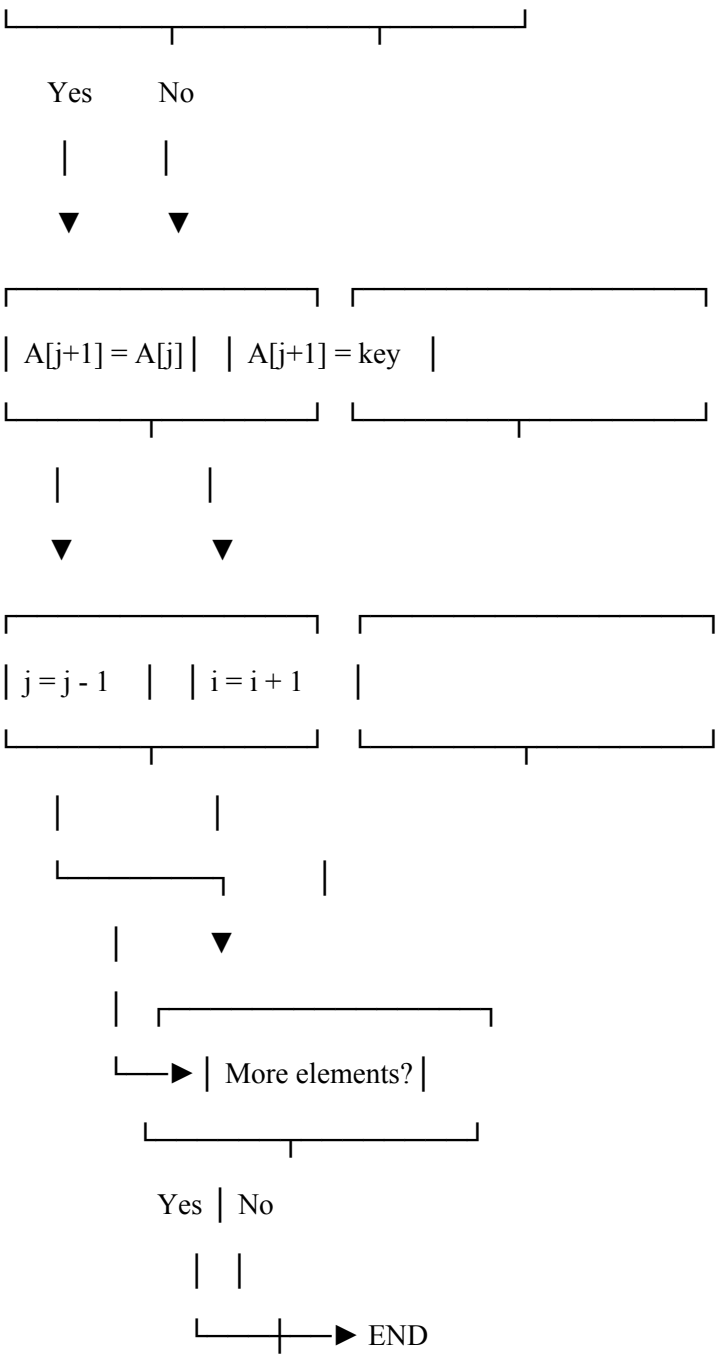
9. Pseudocode

```
INSERTION_SORT(A)


for i = 1 to n - 1


   key = A[i]

   j = i - 1


   while j >= 0 AND A[j] > key


      A[j + 1] = A[j]

      j = j - 1
```

```
    A[j + 1] = key

return A
```

10. Flowchart





11. Step-by-Step Execution

Consider:

[12, 11, 13, 5, 6]

Pass 1

Current:

[12, 11, 13, 5, 6]

Set:

key = 11

j = 0

Compare:

12 > 11

Shift 12:

[12, 12, 13, 5, 6]

Decrease j:

j = -1

Insert original key:

[11, 12, 13, 5, 6]

12. Pass 2

Current:

[11, 12, 13, 5, 6]

Set:

key = 13

j = 1

Compare:

12 > 13 → False

No shifting is required.

Insert key in the same position:

[11, 12, 13, 5, 6]

13. Pass 3

Current:

[11, 12, 13, 5, 6]

Set:

key = 5

j = 2

Compare:

13 > 5

Shift:

[11, 12, 13, 13, 6]

Next:

12 > 5

Shift:

[11, 12, 12, 13, 6]

Next:

11 > 5

Shift:

[11, 11, 12, 13, 6]

Now:

j = -1

Insert original key:

[5, 11, 12, 13, 6]

14. Pass 4

Current:

[5, 11, 12, 13, 6]

Set:

key = 6

j = 3

Compare:

13 > 6 → Shift

12 > 6 → Shift

11 > 6 → Shift

5 > 6 → False

Array becomes:

[5, 5, 11, 12, 13]

Insert key:

[5, 6, 11, 12, 13]

15. Execution Summary

Pass	Key	Shifting Operations	Array After Pass
1	11	12 shifted	[11,12,13,5,6]
2	13	None	[11,12,13,5,6]
3	5	13, 12, 11 shifted	[5,11,12,13,6]
4	6	13, 12, 11 shifted	[5,6,11,12,13]

Final Output

[5, 6, 11, 12, 13]

16. Before vs After Debugging

Feature	Faulty Version	Corrected Version
Key preserved	✖ No	✔ Yes
Larger elements shifted	✖ Incorrectly	✔ Correctly
Key overwritten	✖ Yes	✔ No
Correct insertion	✖ Not guaranteed	✔ Yes
Indexing	Confusing	Consistent
Readability	Low	High
Maintainability	Low	High

Sorting result	Incorrect for some inputs	Correct

17. Optimization Techniques

17.1 Preserve the Key

The most important optimization is to keep:

key = arr[i]

unchanged until insertion.

17.2 Shift Instead of Overwriting the Key

Incorrect

key = arr[j]

Correct

arr[j + 1] = arr[j]

This preserves the value that needs to be inserted.

17.3 Use Consistent Indexing

The algorithm uses:

i → current element

j → previous element

j + 1 → destination for shifted element

This makes the code easier to understand and debug.

17.4 Early Completion for Sorted Data

If:

arr[j] <= key

the while loop stops immediately.

Therefore, already sorted or nearly sorted arrays can be processed efficiently.

18. Performance Evaluation

Test Case 1 – Random Data

Input:

[12, 11, 13, 5, 6]

Output:

[5, 6, 11, 12, 13]

Result:

PASS

Test Case 2 – Already Sorted

Input:

[1, 2, 3, 4, 5]

Output:

[1, 2, 3, 4, 5]

Very few shifts are required.

Result:

PASS

Test Case 3 – Reverse Sorted

Input:

[5, 4, 3, 2, 1]

Output:

[1, 2, 3, 4, 5]

Many shifts are required.

Result:

PASS

Test Case 4 – Duplicate Elements

Input:

[4, 2, 4, 1, 2]

Output:

[1, 2, 2, 4, 4]

Result:

PASS

Test Case 5 – Single Element

Input:

[10]

Output:

[10]

Result:

PASS

19. Test Case Results

Test Case	Input	Expected Output	Actual Output	Status
1	[12,11,13,5,6]	[5,6,11,12,13]	[5,6,11,12,13]	PASS
2	[1,2,3,4,5]	[1,2,3,4,5]	[1,2,3,4,5]	PASS
3	[5,4,3,2,1]	[1,2,3,4,5]	[1,2,3,4,5]	PASS
4	[4,2,4,1,2]	[1,2,2,4,4]	[1,2,2,4,4]	PASS
5	[10]	[10]	[10]	PASS
6	[]	[]	[]	PASS

20. Complexity Analysis

Let:

$$
n = \text{number of elements}
$$

Best Case

The array is already sorted.

For each element, the condition:

arr[j] > key

fails immediately.

Therefore:

$$
\boxed{O(n)}
$$

Average Case

On average, elements need to be shifted through part of the sorted section.

Therefore:

$$
\boxed{O(n^2)}
$$

Worst Case

The array is reverse sorted.

For every new element, all previous elements must be shifted.

Number of shifts:

$$
1+2+3+\cdots+(n-1)
$$

$$
=\frac{n(n-1)}{2}
$$

]

Therefore:

[
\boxed{O(n^2)}
]

21. Space Complexity

Insertion Sort works in-place.

It requires only:

key

i

j

as additional variables.

Therefore:

[
\boxed{O(1)}
]

auxiliary space.

22. Stability Analysis

Insertion Sort is generally **stable**.

The condition:

arr[j] > key

uses a strict greater-than comparison.

Equal elements are not shifted unnecessarily.

Therefore, equal elements retain their relative order.

[
\boxed{\text{Insertion Sort is stable}}
]

23. Correctness Analysis

The corrected algorithm maintains the following invariant:

Before each iteration of the outer loop, the subarray from index 0 to i-1 is sorted.

During each iteration:

1. The current element is stored in key.
2. Larger elements are shifted to the right.
3. The original key remains unchanged.
4. The correct insertion position is identified.
5. The key is inserted.
6. The sorted portion grows by one element.

Therefore, after the final iteration, the complete array is sorted.

24. Problems Encountered and Solutions

Problem	**Cause**	**Solution**
Original key lost	key = arr[j]	Preserve key
Incorrect values inserted	Key changed during shifting	Shift arr[j] instead
Incorrect ordering	Original key unavailable	Store key before loop
Index confusion	Incorrect shift logic	Use arr[j+1] = arr[j]
Poor readability	Insufficient comments	Add explanatory comments

25. Final Clean and Optimized Code

```
def insertion_sort(arr):
    """
    Sort the list using the Insertion Sort algorithm.

    Time Complexity:
        Best Case    : O(n)
        Average Case : O(n^2)
        Worst Case   : O(n^2)
```

```
Space Complexity:
    O(1)

The algorithm sorts the list in-place.
"""

# Start from the second element because
# a single element is already sorted
for i in range(1, len(arr)):

    # Preserve the original element that
    # needs to be inserted
    key = arr[i]

    # Start comparing with the previous element
    j = i - 1

    # Shift larger elements one position to the right
    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    # Insert the preserved key at its correct position
    arr[j + 1] = key

return arr
```

```
# Example execution
arr = [12, 11, 13, 5, 6]

print("Original Array:", arr)

sorted_arr = insertion_sort(arr)

print("Sorted Array:", sorted_arr)
```

Output

Original Array: [12, 11, 13, 5, 6]

Sorted Array: [5, 6, 11, 12, 13]

26. Overall Results

Metric	**Faulty Version**	**Corrected Version**
Original key preserved	✗	✓
Correct shifting	✗	✓
Correct insertion	✗	✓
Sorted output	✗ Not guaranteed	✓
Best-case time	—	**O(n)**
Average-case time	—	**O(n²)**
Worst-case time	—	**O(n²)**
Space complexity	—	**O(1)**
Stable	—	✓ Yes
In-place	—	✓ Yes
Readability	Low	High

Maintainability	Low	High

27. Conclusion

The given Insertion Sort implementation contained a critical debugging error because the statement:

key = arr[j]

overwrote the original value stored in key. The purpose of key is to preserve the element being inserted while larger elements are shifted to the right. Once the key was overwritten, the original element could be lost, resulting in incorrect sorting.

The corrected implementation keeps the original key unchanged and performs the proper shifting operation:

arr[j + 1] = arr[j]

After all larger elements have been shifted, the preserved key is inserted using:

arr[j + 1] = key

The corrected algorithm has:

$$
\boxed{\text{Best Case} = O(n)}
$$

$$
\boxed{\text{Average Case} = O(n^2)}
$$

$$
\boxed{\text{Worst Case} = O(n^2)}
$$

and:

$$
\boxed{\text{Space Complexity} = O(1)}
$$

The debugging exercise demonstrates the importance of **preserving critical variables, maintaining correct indexing, tracing loop execution, and separating the shifting operation from final insertion**. The final implementation is correct, stable, in-place, readable, and suitable for small or nearly sorted datasets.